

ORACLE FUSION HCM FAST FORMULA

Master Notes, Modernized for 2026

Payroll-focused learning guide + interview preparation + practical coding patterns

Prepared from the supplied legacy Fast Formula notes and cross-checked against current Oracle documentation.

Scope: Payroll, Core HR, Absence, Compensation, Time & Labor, Benefits, HCM Extract-related formula patterns

1. What I changed from the original notes

The supplied document is extremely detailed, with approximately 20,691 rendered text lines. Its strongest material is the explanation of contexts, DBIs, input values, arrays, WSA, formula calling, Compensation/TCS examples, and real troubleshooting patterns. The weakness is that several sections are tied to older releases or blog-era terminology. This revision keeps the useful logic but makes the learning path current.

Original material	Assessment	2026 treatment
Contexts / DBIs / Inputs / Returns	Core and still essential	Keep and teach as the foundation
Arrays	Still important	Keep, but emphasize range DBI performance and safe iteration
CHANGE_CONTEXTS	Still valid	Keep, with strong performance warnings
WSA	Still useful	Keep as an advanced pattern, not a default solution
Formula-to-formula calls	Still supported	Update with current SET_INPUT / EXECUTE / GET_OUTPUT / CALL_FORMULA behavior
GET_VALUE_SET	Still supported	Update syntax and explain current use when a DBI isn't available
SET_LOG examples	Legacy / not a reliable current standard	Replace teaching examples with documented logging functions
R11-specific sections	Historical	Move to appendix / label as legacy
EBS / onsite PL/SQL discussion	Historical and potentially misleading for SaaS learners	Replace with current SaaS-safe model
TCS return-string examples	Useful but product/release-specific	Keep as historical pattern and verify current formula-type contract
Formula type lists	Release-sensitive	Teach the selection method instead of memorizing a fixed list

2. The Fast Formula mental model

Think of Fast Formula as a controlled business-rule engine. You do not normally write SQL against the HCM database. Instead, the application invokes the formula with an execution context, exposes approved inputs and database items, and expects specific outputs.

```

APPLICATION PROCESS
  |
  | sets execution context
  v
+-----+
| PERSON_ID / ASSIGNMENT_ID |
| EFFECTIVE_DATE / PAYROLL  |
| other formula contexts    |
+-----+
  |
  v
+-----+
| Fast Formula               |
|                             |
| INPUTS                     |
| DBIs                       |
| local variables            |
| IF / ELSE / LOOP          |
| functions                  |
| CHANGE_CONTEXTS           |
| WSA / child formulas       |
+-----+
  |
  v

```

```

RETURN values
|
v
Calling HCM product uses the result

```

NOTE: The exact contexts, DBIs, input values and return variables are determined by the formula type and the product documentation. Do not choose them from memory just because they worked in another formula.

3. The single most important workflow

1. Understand the business requirement in plain English.
2. Identify where the rule executes: Payroll calculation, element validation, Absence, Compensation, Time & Labor, Benefits, Extract, etc.
3. Select the exact Formula Type required by that setup.
4. Open the formula-type documentation and identify supported contexts, inputs and return values.
5. Choose the smallest set of inputs and DBIs needed.
6. Default DBIs and input values where required.
7. Write the business logic using local variables.
8. Add logging only where it helps troubleshooting.
9. Compile the formula.
10. Attach the formula to the correct setup or process.
11. Test normal, boundary, missing-data and effective-dating scenarios.
12. Validate the payroll/result output, not just compilation.

4. Where do I write the Fast Formula?

For most formula types you create or manage the formula from the Fast Formulas task. Some payroll scenarios also involve editing a generated or related formula from the Elements task. The important question is not only 'where is the editor?' but 'which formula type is invoked by this setup?'

Area	Typical use	What you must identify
My Client Groups > Payroll > Fast Formulas	Create/manage many formula types	Formula Type, LDG, effective start date, text
Payroll > Elements	Element-related generated/custom logic	Which generated formula is being modified and what outputs it expects
Absence setup	Validation, entitlement, payment, proration and related rules	Exact Absence formula type and documented inputs/outputs
Compensation setup	Eligibility, defaults, overrides, TCS items and related rules	Exact Compensation formula type and return contract
Time and Labor	Rule processing and time calculations	OTL/HWM formula type, rule context and logging

NOTE: Oracle's current examples continue to show creation through My Client Groups > Fast Formulas, followed by entering Formula Text and compiling. The exact navigation labels can vary slightly by page/release.

5. Formula header: the code you write first

```

/* DATABASE ITEM DEFAULTS */
DEFAULT FOR PER_ASG_JOB_NAME IS ' '

/* INPUTS */
INPUTS ARE IV_AMOUNT (NUMBER), IV_START_DATE (DATE)

/* LOCAL VARIABLES */

```

```

L_AMOUNT = IV_AMOUNT
L_RESULT = 0

/* BUSINESS LOGIC */
IF L_AMOUNT > 1000 THEN
(
  L_RESULT = L_AMOUNT * 0.10
)
ELSE
(
  L_RESULT = 0
)

/* OUTPUT */
RETURN L_RESULT

```

Memorize the structure, not the exact variable names:

- DEFAULT
- INPUTS
- assignments
- conditions
- functions
- loops
- RETURN

6. Variables: Context vs DBI vs Input vs Local variable

Thing	Who supplies it?	Purpose	Example
Context	Calling application	Defines execution scope / bind-like values	PERSON_ID, EFFECTIVE_DATE
DBI	Formula engine / HCM data	Read application data without SQL	PER_ASG_JOB_NAME
Input	Calling product/setup	Passes explicit values to the formula	IV_START_DATE
Local variable	Your formula	Stores intermediate calculation	L_MONTHLY_PAY
WSA item	Formula execution environment	Shares named values across formulas when supported	WSA_SET('KEY', value)

A useful interview explanation: Context tells the formula 'who/when/which processing scope'. A DBI gives approved HCM data for that scope. An input is data intentionally passed by the calling setup/process. A local variable is your own working memory.

7. Contexts: the SQL-bind-variable analogy

Oracle documents contexts as execution-context values. They act like bind values when database items are fetched. Common examples include EFFECTIVE_DATE, PERSON_ID and PAYROLL_ID. Payroll processing sets the contexts required for the payroll run.

```

L_PERSON_ID = GET_CONTEXT(PERSON_ID, -1)
L_EFF_DATE  = GET_CONTEXT(EFFECTIVE_DATE, '1900/01/01 00:00:00' (DATE))

IF CONTEXT_IS_SET(PERSON_ID) THEN
(
  /* safe logic */
)

```

CHANGE_CONTEXTS is used when the formula needs to evaluate DBIs under another supported context.

```

CHANGE_CONTEXTS (
  PERSON_ID = L_PERSON_ID,
  EFFECTIVE_DATE = L_DATE
)
(
  L_JOB = PER_ASG_JOB_NAME
  L_SAL = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
)

```

NOTE: Do not use CHANGE_CONTEXTS merely because it is available. It can cause DBI values to be fetched again. Combine context changes into as few blocks as practical.

8. Database Items: how to think about DBIs

A DBI is a read-only application data item exposed to Fast Formula. The route behind the DBI determines how its value is obtained. You do not normally need to know the underlying HCM table design.

```

DEFAULT FOR PER_ASG_JOB_NAME IS ' '
DEFAULT FOR PER_ASG_FTE_VALUE IS 1

L_JOB = PER_ASG_JOB_NAME
L_FTE = PER_ASG_FTE_VALUE

IF L_JOB = 'Manager' THEN
(
  L_RESULT = 1
)
ELSE
(
  L_RESULT = L_FTE
)

RETURN L_RESULT

```

DBI selection checklist:

- Is the DBI available to this Formula Type?
- Does the required context exist for the DBI?
- Is the DBI scalar or array/range?
- What is its data type?
- Does it need a default?
- Will referencing it cause an unnecessary database fetch?
- Can the value be passed as an INPUT instead?

9. INPUTS: often the performance-friendly choice

Oracle's current performance guidance recommends INPUTS instead of DBIs whenever practical because input variables don't require a database fetch in the same way DBIs do.

```

INPUTS ARE IV_ANNUAL_SALARY (NUMBER), IV_FTE (NUMBER)

L_SALARY = IV_ANNUAL_SALARY
L_FTE = IV_FTE

IF L_FTE <= 0 THEN
(
  L_FTE = 1
)

L_MONTHLY = (L_SALARY * L_FTE) / 12

RETURN L_MONTHLY

```

10. DEFAULT FOR and WAS DEFAULTED

Fast Formula doesn't use SQL NULL handling in the same way as PL/SQL. Formula design commonly uses DEFAULT declarations and tests whether a value was defaulted.

```

DEFAULT FOR PER_ASG_FTE_VALUE IS 1
DEFAULT FOR CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT IS 0

L_RESULT = 0

IF CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS NOT DEFAULTED THEN
(
  L_RESULT = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
)

RETURN L_RESULT

```

NOTE: Choose a default that is safe for the business logic. A default of 0, -1, blank text, or a sentinel date has different meanings. Document why you selected it.

11. Core coding patterns you must master

11.1 IF / ELSE

```

IF L_GRADE >= 10 THEN
(
  L_BONUS = L_SALARY * 0.20
)
ELSE
(
  L_BONUS = L_SALARY * 0.10
)

```

11.2 Multiple conditions

```

IF L_STATUS = 'ACTIVE' AND L_FTE > 0 THEN
(
  L_ELIGIBLE = 'Y'
)
ELSE
(
  L_ELIGIBLE = 'N'
)

```

11.3 Nested conditions

```

IF L_COUNTRY = 'IN' THEN
(
  IF L_SALARY > 100000 THEN
    (
      L_RATE = 0.20
    )
  ELSE
    (
      L_RATE = 0.10
    )
  )
)

```

11.4 WHILE loop

```

I = 1
L_TOTAL = 0

WHILE L_ARRAY.EXISTS(I) LOOP
(
  L_TOTAL = L_TOTAL + L_ARRAY[I]
  I = I + 1
)

```

```
RETURN L_TOTAL
```

11.5 EXIT from a loop

```
I = L_ARRAY.FIRST(-1)

WHILE L_ARRAY.EXISTS(I) LOOP
(
  IF L_ARRAY[I] = 'TARGET' THEN
  (
    L_FOUND = 'Y'
    EXIT
  )

  I = L_ARRAY.NEXT(I, -1)
)
```

12. Arrays: the part that separates beginners from strong consultants

Current Oracle documentation supports arrays such as DATE_NUMBER, NUMBER_NUMBER, TEXT_NUMBER and related combinations. Array DBIs are commonly range-style data sets. The correct pattern is to default the array, get an index, check EXISTS, read the indexed value, then move to FIRST/NEXT or another appropriate index operation.

```
DEFAULT FOR MY_ARRAY IS EMPTY_TEXT_NUMBER

I = MY_ARRAY.FIRST(-1)

WHILE MY_ARRAY.EXISTS(I) LOOP
(
  L_VALUE = MY_ARRAY[I]

  /* business logic */

  I = MY_ARRAY.NEXT(I, -1)
)
```

NOTE: Do not iterate a range DBI blindly. First understand what each index represents, whether companion arrays use the same index, and how much data the DBI can return.

13. CHANGE_CONTEXTS: the effective-dating engine inside your logic

This is especially important in HCM because employee data is effective-dated. A requirement such as 'salary as of the beginning of each month' may require changing EFFECTIVE_DATE while iterating through months.

```
L_DATE = L_PERIOD_START

WHILE L_DATE <= L_PERIOD_END LOOP
(
  CHANGE_CONTEXTS(EFFECTIVE_DATE = L_DATE)
  (
    IF CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS NOT DEFAULTED THEN
    (
      L_MONTHLY = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT / 12
    )
  )

  L_DATE = ADD_MONTHS(L_DATE, 1)
)
```

Before using this pattern, ask:

- What date should the DBI actually represent?
- Is EFFECTIVE_DATE already set correctly by the calling process?
- Do I need DATE_EARNED or another payroll date instead?
- Will each context change cause repeated DBI retrieval?
- Can I reduce the number of context changes?

14. Formula calling: parent and child formulas

Current Oracle documentation supports modular formula calls. The separate-call pattern is SET_INPUT → EXECUTE → GET_OUTPUT. CALL_FORMULA can package the same sequence into a self-contained call.

```
/* Parent formula */
IF IS_EXECUTABLE('XX_CHILD_FORMULA') THEN
(
  SET_INPUT('IV_RATE', L_RATE)
  SET_INPUT('IV_HOURS', L_HOURS)

  EXECUTE('XX_CHILD_FORMULA')

  L_AMOUNT = GET_OUTPUT('O_AMOUNT', 0)
)

RETURN L_AMOUNT
```

Why use child formulas?

- Reuse common calculations.
- Separate legislation-specific logic where appropriate.
- Isolate logic requiring a different Formula Type and therefore different DBIs/contexts.
- Make large formulas easier to test and maintain.

NOTE: The child formula must be executable, valid for the effective date, and compatible with the calling formula's restrictions. Contexts are inherited, but explicitly set inputs/contexts can be passed.

15. GET_VALUE_SET: controlled lookup when a DBI isn't available

Current Oracle documentation describes GET_VALUE_SET as a way to retrieve a value from a configured value set, particularly when a suitable DBI isn't available. The value set configuration and bind syntax must match the current product rules.

```
L_PERSON_ID = GET_CONTEXT(PERSON_ID, -1)

L_BIND =
  '|=PERSON_ID=' || TO_CHAR(L_PERSON_ID) || ''

L_RESULT =
  GET_VALUE_SET('XX_PERSON_LOOKUP_VS', L_BIND)

RETURN L_RESULT
```

NOTE: Treat value sets as a controlled data-access configuration, not as a license to write arbitrary SQL inside Fast Formula. Validate the value set's table/columns, validation type, aliases and bind behavior against your current environment.

16. Working Storage Area (WSA)

WSA provides named storage that can allow formulas to share values. The original notes explain this well, but WSA should be considered an advanced mechanism rather than the first solution.

```
L_SALARY = 0

IF CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT WAS NOT DEFAULTED THEN
(
  L_SALARY = CMP_ASSIGNMENT_SALARY_ANNUAL_AMOUNT
)

WSA_SET('XX_SALARY', L_SALARY)

/* Later formula */
IF WSA_EXISTS('XX_SALARY', 'NUMBER') THEN
(
  L_SALARY = WSA_GET('XX_SALARY', 0)
  WSA_DELETE('XX_SALARY')
)
```

Interview point: WSA is not the same as a local variable. A local variable lives inside the current formula execution; WSA is intended for named values shared across formulas in supported execution scenarios.

17. Logging and debugging: modern approach

The original notes contain many SET_LOG examples. For current learning, do not copy SET_LOG blindly. Oracle documents ESS_LOG_WRITE for Enterprise Scheduler logs and application-specific logging functions such as PAY_INTERNAL_LOG_WRITE in payroll examples. Use the logging mechanism documented for the formula/process you are troubleshooting.

```
L_LOG = ESS_LOG_WRITE(
  'XX_FORMULA START - Person=' || TO_CHAR(L_PERSON_ID)
)

L_LOG = ESS_LOG_WRITE(
  'Salary=' || TO_CHAR(L_SALARY)
)

L_LOG = ESS_LOG_WRITE(
  'Result=' || TO_CHAR(L_RESULT)
)
```

Debugging sequence:

- Confirm the seeded behavior before changing the custom formula.
- Test with hard-coded values to isolate formula mechanics.
- Replace complicated functions temporarily with constants.
- Log key inputs, contexts and intermediate values.
- Check the process log or application-specific rule-processing log.
- Verify the final payroll/absence/compensation result.
- Do not solve a compilation error by randomly changing DBIs or return variables.

18. Performance rules you should memorize

- Use INPUTS instead of DBIs when the calling process can provide the value.
- Reference DBIs only when needed.
- Use meaningful local variables or aliases for long DBI names.
- Avoid unnecessary CHANGE_CONTEXTS.

- Combine context changes into one block where practical.
- Exit WHILE loops as soon as the required item is found.
- Avoid expensive array traversal when a scalar input or DBI can satisfy the requirement.
- Keep logging limited in high-volume payroll runs.
- Avoid doing the same calculation repeatedly in multiple branches.
- Test performance with realistic payroll populations, not one employee.

19. Payroll-specific thinking

For Payroll, start with the payroll event and the element, not with the code. A payroll requirement usually becomes one of four questions:

- Should this element process? → skip/control logic.
- Is the entered value valid? → validation/defaulting logic.
- What amount/result should be calculated? → calculation formula.
- How should the result be prorated or adjusted for an event? → proration or related formula type.

20. Payroll calculation example: a clean mental model

Example requirement: Monthly allowance = eligible days × daily rate, but cap the result at 5,000.

```
/* Inputs supplied by the formula type / element setup */
INPUTS ARE IV_ELIGIBLE_DAYS (NUMBER), IV_DAILY_RATE (NUMBER)

L_DAYS = IV_ELIGIBLE_DAYS
L_RATE = IV_DAILY_RATE
L_AMOUNT = L_DAYS * L_RATE

IF L_AMOUNT > 5000 THEN
(
  L_AMOUNT = 5000
)

RETURN L_AMOUNT
```

The coding is easy. The consultant work is identifying whether IV_ELIGIBLE_DAYS and IV_DAILY_RATE are actually available as inputs for the selected Formula Type, and whether the required return variable is really the numeric result you think it is.

21. Payroll interview scenario: mid-month hire

Requirement: Monthly salary is 60,000. Employee joins on the 16th. Calculate salary for a 30-day month.

```
INPUTS ARE IV_MONTHLY_SALARY (NUMBER),
           IV_DAYS_IN_PERIOD (NUMBER),
           IV_ELIGIBLE_DAYS (NUMBER)

L_DAILY_RATE = IV_MONTHLY_SALARY / IV_DAYS_IN_PERIOD
L_PAY = L_DAILY_RATE * IV_ELIGIBLE_DAYS

RETURN L_PAY
```

Interview discussion:

- Where does the Formula Type get IV_ELIGIBLE_DAYS?
- Is the payroll period 30 calendar days or another proration basis?
- Does the element's proration configuration already calculate the eligible days?

- Should the formula use payroll dates, date earned, or a supplied period?
- How are leap years and partial periods handled?
- What happens for termination in the same period?

22. Fast Formula syntax cheat sheet

Pattern	Syntax	Use
Default DBI	DEFAULT FOR X IS 0	Safe fallback
Input	INPUTS ARE X (NUMBER), D (DATE)	Explicit data from caller
Assignment	L_TOTAL = L_A + L_B	Store calculation
Condition	IF X > 0 THEN (...) ELSE (...)	Business rule
Boolean	IF A = 'Y' AND B > 0 THEN	Combine rules
Function	ADD_MONTHS(D, 1)	Date calculation
Context	GET_CONTEXT(PERSON_ID, -1)	Read execution context
Context change	CHANGE_CONTEXTS(EFFECTIVE_DATE = D) (...)	Evaluate DBIs at another context
Array	A[I]	Read array element
Array existence	A.EXISTS(I)	Safe iteration
Formula call	SET_INPUT → EXECUTE → GET_OUTPUT	Reuse logic
Return	RETURN L_RESULT	Send result to caller

23. How to write a formula from scratch: your reusable template

```

/*
Formula Name : XX_CUSTOM_RULE
Formula Type : <EXACT FORMULA TYPE>
Purpose      : <ONE SENTENCE BUSINESS PURPOSE>
*/

/* 1. Defaults */
DEFAULT FOR <DBI_1> IS <SAFE_DEFAULT>

/* 2. Inputs */
INPUTS ARE <INPUT_1> (NUMBER),
          <INPUT_2> (DATE)

/* 3. Contexts */
L_PERSON_ID = GET_CONTEXT(PERSON_ID, -1)
L_EFF_DATE  = GET_CONTEXT(EFFECTIVE_DATE,
                          '1900/01/01 00:00:00' (DATE))

/* 4. Initialize outputs */
L_RESULT = 0

/* 5. Read/validate data */
IF <DBI_1> WAS NOT DEFAULTED THEN
(
  /* 6. Business logic */
  IF <CONDITION> THEN
  (
    L_RESULT = <CALCULATION>
  )
)

/* 7. Optional debugging */
L_LOG = ESS_LOG_WRITE(
  'XX_CUSTOM_RULE result=' || TO_CHAR(L_RESULT)
)

```

```
/* 8. Return EXACT required variable(s) */
RETURN L_RESULT
```

24. The consultant's debugging tree

```

COMPILE ERROR?
|
+-- Syntax / datatype?
|   -> check INPUTS, DEFAULTS, operators, function signatures
|
+-- DBI not recognized?
|   -> check Formula Type + supported contexts + exact DBI name
|
+-- Formula compiles but setup doesn't show it?
|   -> check Formula Type, LDG, effective date, attachment
|
+-- Formula runs but result is wrong?
|   -> log contexts + inputs + DBIs + intermediate values
|
+-- Wrong historical result?
|   -> inspect EFFECTIVE_DATE / DATE_EARNED / CHANGE_CONTEXTS
|
+-- Works for one employee but fails at scale?
|   -> inspect DBIs, arrays, CHANGE_CONTEXTS, logging and loops

```

25. Common mistakes from the original notes that I would avoid

- Copying a TCS example into Payroll simply because both use Fast Formula.
- Assuming a DBI is available because it exists somewhere in HCM.
- Assuming a return variable name is interchangeable across Formula Types.
- Using an old logging function from a blog without checking the current product documentation.
- Treating R11-specific array workarounds as the current standard.
- Using CHANGE_CONTEXTS repeatedly inside nested loops without considering DBI retrieval cost.
- Writing a formula first and only later figuring out which Formula Type invokes it.
- Testing only the happy path.
- Forgetting effective dating.
- Changing or deleting a production formula without understanding its existing attachments and effective-dated behavior.

26. Interview questions you should be able to answer

- What is Fast Formula and where is it used in Oracle Fusion HCM?
- What is the difference between a Formula Type and a Fast Formula?
- What is a context? Give three examples.
- What is a DBI? Why can't every DBI be used in every formula?
- Difference between a DBI and an INPUT?
- Why do we use DEFAULT FOR?
- What does WAS NOT DEFAULTED mean?
- When would you use CHANGE_CONTEXTS?
- Why can CHANGE_CONTEXTS hurt performance?
- What are array DBIs and how do you loop through them?
- What is WSA and when would you use it?
- How do you call one Fast Formula from another?
- Difference between SET_INPUT, EXECUTE and GET_OUTPUT?
- What is IS_EXECUTABLE used for?

- How do you troubleshoot a formula that compiles but returns the wrong value?
- Why is effective dating important in Fast Formula?
- How do you decide whether a requirement needs a Payroll calculation formula, validation formula, skip formula or proration logic?
- What must you verify before using a DBI?
- How do you optimize a formula used for thousands of employees?
- What is the difference between formula compilation and business-result validation?

27. Recommended learning sequence for you

13. Syntax and variables
14. Formula Type and return contract
15. Contexts
16. DBIs and defaults
17. INPUTS
18. IF / ELSE and boolean logic
19. Dates and date arithmetic
20. Arrays
21. CHANGE_CONTEXTS
22. Payroll element calculation patterns
23. Proration
24. Formula-to-formula calls
25. GET_VALUE_SET
26. WSA
27. Logging and debugging
28. Performance tuning
29. Real payroll scenarios: hire, termination, transfer, salary change, retro, absence, overtime, deductions

28. Practical exercises

Beginner: Return 10% of an input amount if amount > 10,000.

Beginner: Return Y if an employee is active and FTE > 0.

Intermediate: Calculate monthly salary from annual salary and FTE.

Intermediate: Calculate a daily amount using supplied period days.

Intermediate: Apply a 5,000 cap.

Intermediate: Use EFFECTIVE_DATE and a DBI safely.

Advanced: Loop through an array DBI and find a matching value.

Advanced: Calculate a result for each month using CHANGE_CONTEXTS.

Advanced: Call a child formula with SET_INPUT / EXECUTE / GET_OUTPUT.

Advanced: Use GET_VALUE_SET to retrieve a controlled business mapping.

Expert: Design a payroll formula that handles hire, termination, salary change and missing input without unnecessary DBI calls.

29. Current Oracle documentation checkpoints

Use the current Oracle documentation as the source of truth for exact Formula Type contracts, contexts, DBIs, inputs, outputs and release-specific behavior. The supplied notes are a learning resource, not a replacement for the documentation of the formula type in your tenant.

- **Overview of Writing Formulas:**
<https://docs.oracle.com/en/cloud/saas/human-resources/oapff/overview-of-writing-formulas.html>
- **Formula Contexts:** <https://docs.oracle.com/en/cloud/saas/human-resources/oapff/formula-contexts.html>
- **Improve Formula Performance:**
<https://docs.oracle.com/en/cloud/saas/human-resources/oapff/improve-formula-performance.html>
- **Overview of Fast Formula Functions:**
<https://docs.oracle.com/en/cloud/saas/human-resources/oapff/overview-of-fast-formula-functions.html>
- **Miscellaneous Formula Functions:**
<https://docs.oracle.com/en/cloud/saas/human-resources/oapff/miscellaneous-formula-functions.html>
- **Formula Debugging and Troubleshooting:**
<https://docs.oracle.com/en/cloud/saas/human-resources/oapff/formula-debugging-and-troubleshooting.html>
- **Overview of Calling a Formula from a Formula:**
<https://docs.oracle.com/en/cloud/saas/human-resources/oapff/overview-of-calling-a-formula-from-a-formula.htm>
 |
- **Formula FAQ:** <https://docs.oracle.com/en/cloud/saas/human-resources/oapff/formula-faq.html>

30. Final rule to remember

Do not start with code. Start with the execution contract. Ask: What process invokes this formula? Which Formula Type? What contexts are set? What inputs are passed? Which DBIs are legal? What exact output is expected? Only then write the business logic. This single habit prevents a large percentage of Fast Formula defects.

Source note: This document modernizes the supplied “Oracle HCM Fusion Fast Formula notes.docx”. Legacy examples are intentionally simplified or reframed where they are release-specific. Always validate exact syntax and formula contracts against your current Oracle HCM release and tenant configuration.